Module	Problem	Candidates (time complexity)	Selection	Justification
A. Relief inventory	n crates with weight w_i (1-20 kg) and value v_i, capacity W. Choose indivisible crates with total weight <= W and maximum total value (0/1 knapsack, NP-hard). Ours: n <= 3,000, W <= 3,631.	- DP table over (crate, capacity): O(nW) time and space (pseudo-polynomial: polynomial in the value W, exponential in its number of bits). - Greedy by v/w: O(n log n), no guarantee. - Exhaustive search: $O(2^n n)$.	**Primary:** DP table (set recovered by traceback). **Baseline:** greedy v/w; exhaustive search on tier S (= cargo_opt).	Weights are small integers and W is small: nW <= $5.3x10^6$ cells on tier L, so the table fits. Greedy is unsafe: 114 of 200 (57%) in trap 47, not optimal in 13 tier-S cases. Assumes integer weights. Fails if weights were real numbers or W huge (10^9): use branch-and-bound or an approximation scheme.
B. Route planning	V locations, E roads (E <= 3V); step u->v costs minutes + penalty(v) >= 1; closed roads removed; source = HQ. Output: cheapest route and cost (or none), the reachable set, a fewest-roads route.	- Dijkstra with binary heap: O((V + E) log V). - Bellman-Ford: O(VE), allows negative costs. - BFS: O(V + E), unweighted questions only (reachability, fewest roads).	**Primary:** Dijkstra for cost; BFS for reachable set and fewest-roads route. **Baseline:** Bellman-Ford (same answers, slower).	All step costs are positive - exactly Dijkstra's assumption; the graph is sparse, so a heap beats the $O(V^2)$ array version. Cost depends on direction (penalty of the entered node), so each road becomes two arcs. Trap 48: the 2-road route costs 126, the cheapest 3, so BFS cannot replace Dijkstra. Fails with negative costs (e.g. rewards): use Bellman-Ford.
C. Rescue scheduling	n windows [start, finish) with value; touching windows are compatible. Output: largest compatible set, or compatible set of maximum total value. n <= 3,000.	- Earliest-finish greedy: O(n log n). - Weighted interval DP (sort by finish, binary search for the last compatible window): O(n log n). - Exhaustive: $O(2^n n)$. - Greedy by value: O(n log n), not optimal.	**Primary:** count: earliest-finish greedy; value: weighted interval DP. **Baseline:** exhaustive on tier S (claims); greedy by value.	Exchange argument: some optimal set starts with the window that finishes first, so greedy is exact for count; with values it fails (one long valuable window beats two short ones), hence DP. Cyclone windows overlap 2+ others on average, so choices really conflict. Assumes one team, no travel time. Fails with k parallel teams or travel time between windows.
D. Emergency queue	n alarms (tick, priority 1-10); each tick releases one alarm, the most urgent present (ties: earlier tick, then alarm_id). Output: release order. n <= 5,000.	- Binary heap: O(log n) per insert / extract, O(n log n) total. - Unsorted list, scan for the maximum: O(n) per release, $O(n^2)$ total. - Bucket queue (10 FIFO lists): O(1) per operation.	**Primary:** binary heap with key (-priority, tick, alarm_id). **Baseline:** linear scan.	Arrivals (1.2-1.8 per tick) exceed the single release, so 17-44% of alarms are still waiting at the end; a scan then costs about 10^6-10^7 steps on tier L, the heap about $6x10^4$. The bucket queue is faster only because there are exactly 10 levels. Fails if priorities change while waiting (ageing): needs decrease-key or re-insertion.
E. Radio logs	Log of length L <= 10^5 over 38 symbols; 6 patterns of length m <= 40; reports of length p, q <= 1,000. Output: (1) all start positions, overlaps included; (2) minimum number of edits and an edit script; (3) prefix-free binary code of minimum total length.	(1) KMP: O(L + m); naive: O(Lm); Rabin-Karp: O(L + m) expected, O(Lm) worst. (2) DP table: O(pq) time and space; plain recursion: $O(3^{p+q})$. (3) Huffman with min-heap: O(L + s log s), s <= 38 symbols; fixed 6-bit code: O(L).	**Primary:** (1) KMP, (2) edit-distance DP with traceback, (3) Huffman. **Baseline:** naive scan; recursion on tier S; fixed 6-bit code.	Trap 49: 98% of the log is one letter and each pattern is that letter m-1 times plus another symbol, so naive matching makes about m comparisons per position (about Lm), while KMP never re-reads the log. pq <= 10^6, so the full table (needed for the script) fits. Huffman is optimal among prefix codes; beacon logs (one symbol >= 25%) gain most. Fails: a one-symbol log needs a special 1-bit code; much longer reports would need Hirschberg's linear-space method.
F. Network planning	Network after closures, cable cost per road (1-100). Output: roads linking all locations reachable from the HQ with minimum total cable (minimum spanning tree of the HQ's component).	- Prim with binary heap: O(E log V). - Kruskal with union-find: O(E log E). - Prim with an array: $O(V^2)$.	**Primary:** Prim grown from the HQ. **Baseline:** Kruskal on the same component (equal totals = cross-check).	Prim started at the HQ only reaches the HQ's component, which is exactly "all reachable locations", so closures need no extra step. E <= 3V is sparse, so the heap version beats the $O(V^2)$ array. Assumes undirected roads. Fails if every component were wanted (spanning forest): Kruskal gives that directly; for dense graphs the array version is better.
G. Chains and cuts	(1) Chain of n matrices, dims $d_0..d_n$, n <= 300: bracketing with fewest scalar multiplications. (2) Drum of length L <= 2,000, price p(j) for a piece of length j: cutting with maximum total price.	(1) Interval DP: $O(n^3)$ time, $O(n^2)$ space; all bracketings: Catalan number, Omega($4^n/n^{1.5}$); left-to-right order: O(n), no guarantee. (2) Rod-cutting DP: $O(L^2)$; all cuttings: $O(2^{L-1})$; greedy by price per unit length: $O(L^2)$, not optimal.	**Primary:** (1) matrix-chain DP, (2) rod-cutting DP. **Baseline:** left-to-right cost; exhaustive on tier S (drum_opt); greedy by price per length.	Both have optimal substructure: the best bracketing splits at some k, the best cutting starts with some first piece. $n^3/6 = 4.5x10^6$ and $L^2 = 4x10^6$ steps are cheap. Our prices are noisy and capped at 500, so greedy by density is misleading. Fails if every cut had a cost or the number of pieces were limited: an extra DP dimension is needed.
H. Hard planning	(1) 3-SAT: n <= 50 switches, m <= 6n rules. (2) Subset sum: n <= 100 values <= 1,000, target T. (3) TSP: n sites, Manhattan distance; n <= 16 on tiers S, M, 200-500 on L, X. (4) Vertex cover: n <= 400 nodes, m edges, cover of at most k nodes? All four are NP-hard.	(1) DPLL backtracking + unit propagation: $O(2^n m)$ worst; brute force: always $2^n m$; WalkSAT: cannot prove "none". (2) DP over reachable sums: O(nT); backtracking: $O(2^n)$; meet in the middle: $O(2^{n/2} n)$. (3) Held-Karp: $O(n^2 2^n)$ time, $O(n2^n)$ space; brute force: O(n!); nearest neighbour $O(n^2)$ + 2-opt $O(n^2)$ per pass. (4) Bounded search tree: $O(2^k(n + m))$; maximal-matching 2-approximation: O(n + m); brute force: O(C(n,k) m).	**Primary:** (1) DPLL, (2) subset-sum DP, (3) Held-Karp for n <= 16, NN + 2-opt on L/X, (4) bounded search tree with degree-1 and degree->k reductions. **Baseline:** brute force on tier S; planted tour_length and NN tour; matching bound.	(1) 2^{50} rules out brute force; propagation prunes, but 9 of our M/L cases have 3.5-4.8 rules per switch, near the hardest ratio (about 4.26), so a time limit is needed. (2) nT <= $2x10^6$: fast; fails for values like 10^{12}. (3) Held-Karp is exact up to 16 sites ($1.7x10^7$ steps) but impossible for 500; 2-opt has no guarantee, so we compare with tour_length. (4) 40-191 sensors per L case have one edge, so the degree-1 rule fixes much of the cover; |matching| > k proves "none". Fails for k near 100 with few reductions.
I. Nearest sites and checksums	(1) n <= 500 sites, integer x, y in 0-999: pair at minimum squared Euclidean distance. (2) Product of two long decimal numbers of d digits built from the 8-digit serials (d <= 8n).	(1) Divide and conquer (split by x, check at most 7 neighbours in the strip): O(n log n); brute force over all pairs: $O(n^2)$. (2) Karatsuba: $O(d^{\log_2 3}) = O(d^{1.585})$; grade-school: $O(d^2)$.	**Primary:** closest-pair divide and conquer; Karatsuba on digit lists. **Baseline:** brute force pairs; grade-school multiplication.	Squared distances keep everything in exact integers. At n = 500 brute force ($1.2x10^5$ pairs) is still fast, so the gain shows in timing plots rather than feasibility. Python's built-in integer product already uses Karatsuba, so we use it only to verify. Fails (is slower) on short numbers because of recursion overhead: below about 32-64 digits switch to grade-school.

Member	Contribution
Hassan Tariq 24i-2555 Section: __ Group 40	**Data modules:** generator core (config.py, main.py, per-case and per-module seeding), crates.py (category rules, capacity, traps 46 and 47), scheduling.py (windows, touching pairs, cyclone overlap; alarms, alarm rates, earthquake bursts). **Validator checks:** V1 (sheets, headers, cell types, sort order, case ids, tier and scenario) and V2 (all sizes and value ranges). **Claims:** cargo_opt, window_count_opt, window_value_opt; variety rule 1. **Approach note:** Part 1 (from story to numbers). **Proposed decision modules for Iteration 2:** A (knapsack DP), C (interval scheduling), D (emergency queue).
Noah Faraz 24i-2585 Section: __ Group 40	**Data modules:** network.py (six network shapes, locations and penalties, closure waves, landslide cut, trap 48), radio.py (story-based log, storm noise and beacons, patterns, report pairs with edit_bound, trap 49). **Validator checks:** V3 (identifiers and references), V4 (connectivity, waves), V5 (pattern counts, edit distance, log parts). **Claims:** variety rule 2 (fewest-roads versus cheapest routes). **Approach note:** Part 3 (the traps). **Proposed decision modules for Iteration 2:** B (Dijkstra and BFS), E (KMP, edit distance, Huffman), F (Prim and Kruskal).
Diya Hurmat 24i-0094 Section: __ Group 40	**Data modules:** planning.py (sites and planted tour, pipeline, drum prices, sensor graph and planted cover, safety rules and planted assignment, rations and planted selection), safety-ratio bands (variety rule 3). **Validator checks:** V6 (planted objects), V7 (signatures and variety rules), V8 (traps); the broken copies in validator/tests and validation_report.txt. **Claims:** drum_opt, tour_opt, cover_min. **Approach note:** Parts 2 and 4. **Proposed decision modules for Iteration 2:** G (matrix chain, rod cutting), H (DPLL, subset-sum DP, Held-Karp / 2-opt, vertex cover), I (closest pair, Karatsuba).